

2-2. 객체

자바스크립트는 객체(Object)기반의 프로그래밍 언어로 원시 값을 제외한 나머지 (함수, 배열, 정규 표현식 등) 모두 객체입니다.

- 원시 값은 변경 불가능한 값으로 pass by value
- 객체타입의 값, 즉 객체는 변경 가능한 값으로 pass by reference
 - 값 자체를 변경 합니다.
 - 변수는 재할당이 가능합니다.
 - name/value 쌍으로 된 Property로 구성 되기 때문에 Map 처럼 사용 될 수 있음
 - {}로써 이용 해서 생성을 하며 속성 값에 접근 할 때 키로 접근
 - 객체의 속성을 사용 하기 위해서 for in 반복문을 사용

1. Property와 Method

자바스크립트에서 사용할 수 있는 모든 값은 프로퍼티 값이 될 수 있고, 자바스크립트의 함수는 일급 객체이므로 값으로 취급이 가능합니다.

- Property : 객체의 상태를 나타내는 값(Data)으로 JavaScript에서 제공 하는 모든 자료형을 가질 수 있음
- Method : 프로퍼티(상태 데이터)를 참조하고 조작할 수 있는 동작(Behavior)

2. 객체 생성

- 객체 리터럴 : 가장 대중적인 방법
- Object 생성자 함수
- 생성자 함수
- Object.create 메서드
- 클래스(ES6)

2-1. 객체 리터럴 : 가장 대중적인 방법

객체를 생성하기 위해 클래스를 먼저 정의하고 new 연산자와 함께 생성자를 호출할 필요가 없다.

- {} 로 감싸고 key:object를 쉼표(,)로 구분
- 생성한 객체는 언제라도 변경 가능 하며, 내장 네이티브 객체의 프로퍼티도 대부분 변경이 가능 하다.
- 직접접근 : fun.name;
- 문자로접근 : fun['name'];
- 변수로 접근
 - var get = 'name';
 - fun[get]

```
const 객체명 = {} // 빈 객체
```

```
const 객체명 = {  
  프로퍼티1 : 값1,  
  프로퍼티2 : 값2,  
  프로퍼티3 : 값3,
```

```

    프로퍼티4 : 값4,
    ...
    메소드명 : function(){
        수행할 문장
    },
    ...
}

```

```

// 객체 생성
const person = {};

// 객체 속성 추가
person.name = '홍길동';
person.getName = function() {
    return person.name;
};

console.log("person :: ", person.getName()); // person :: 홍길동

// 함수 재 정의
person.getName = function() {
    return "홍당무";
};

console.log("person :: ", person.getName()); // person :: 홍당무

```

객체 생성시 프로퍼티 생성 예제

```

const person = {
    name : '홍길동',
    getName : function () {
        return this.name;
    }
};

console.log("person ::", person.getName()); // person :: 홍길동
person.name = "홍당무";
console.log("person ::", person.getName()); // person :: 홍당무

```

2-1-1 배열 리터럴

```

var a = ["1","2","3"];
// 배열도 객체 이므로 object
console.log(typeof a); // object
console.log(a.constructor); // [Function: Array]

```

```
var aa = new Array(3);
console.log(typeof aa); // object
console.log(aa.constructor); // [Function: Array]
console.log(aa.length); // 3
console.log(aa[0]); // undefined

var bb = [3];
console.log(bb.length); // 1
console.log(bb[0]); // 3
```

2-1-2. 객체 리터럴의 확장 기능

문자열 또는 문자열로 타입 변환할 수 있는 값으로 평가되는 표현식을 사용해 프로퍼티 키를 동적으로 생성할 수도 있다

```
const prefix = "prop";
let index = 0;

let obj = {};

obj[prefix + "-" + ++index] = index;
obj[prefix + "-" + ++index] = index;
obj[prefix + "-" + ++index] = index;

console.log(obj); // {prop-1: 1, prop-2: 2, prop-3: 3}
```

2-2. Object 생성자 함수

new 연산자와 함께 Object 생성자 함수 생성하면 빈 객체를 반환합니다.

```
// 객체 생성
const customer = new Object();

// 프로퍼티 추가
customer.name = "홍길동"
customer.takeName = function() {
  console.log(this.name);
};

console.log(customer); // {name: '홍길동', takeName: f}
customer.takeName();   // 홍길동 :: 함수를 통한 접근
console.log(customer.name); // 홍길동 :: 직접 접근
```

생성자에 파라미터로 전달 시 해당 값의 형으로 결정 되므로 객체 생성시 new 연산자를 사용 하지 말아야 합니다.

```
const personObj = new Object();
console.log(personObj.constructor === Object); // true

// new로 생성시 파라미터에 의해서 타입이 결정 된다.
const personObj01 = new Object(1);
console.log(personObj01.constructor === Object); // false
console.log(personObj01.constructor === Number); // true
```

2-3. 생성자 함수

동일한 프로퍼티를 갖는 객체를 여러 개 생성해야 하는 경우 객체 리터럴은 매번 같은 프로퍼티를 코드로 작성해야 합니다. 이것은 객체의 재 사용을 떨어뜨리므로 인스턴스 생성을 하여 동일 프로퍼티 구조로 여러개를 만들기 위해 생성자 함수를 사용합니다.

리터럴 표기법과 달리 생성자를 활용하면 '='연산자로 표현해야 합니다.

```
function 생성자명(매개변수1, 매개변수2 ...){
    this.프로퍼티1 = 매개변수값;
    this.프로퍼티2 = 매개변수값;
    ....
}
const 객체명 = new 생성자명(값1, 값2,...); //새로운 주소값을 가진 객체를 만들
```

```
function cust(name, ages) {
    // 1. 암묵적으로 빈 객체 생성되고 this 바인딩

    // 2. this 바인딩되어 있는 인스턴스 초기화
    this.name = name;
    this.ages = ages;
    this.fnCust00 = () => {
        return `fnCust00 :: ${this.name}님 나이는 ${this.ages} 입니다`;
    };
    this.fnCust01 = function() {
        return `fnCust01 :: ${this.name}님 나이는 ${this.ages} 입니다`;
    };

    // 3. 암묵적으로 this 반환
    // 4. 명시적으로 원시 값을 반환하면 해당 값은 무시되고 this 반환
    return "고객";
}

// 5. 인스턴스 생성- 명시적으로 반환한 객체 반환
const customer00 = new cust('홍길동', 30);
const customer01 = new cust('김길고', 50);

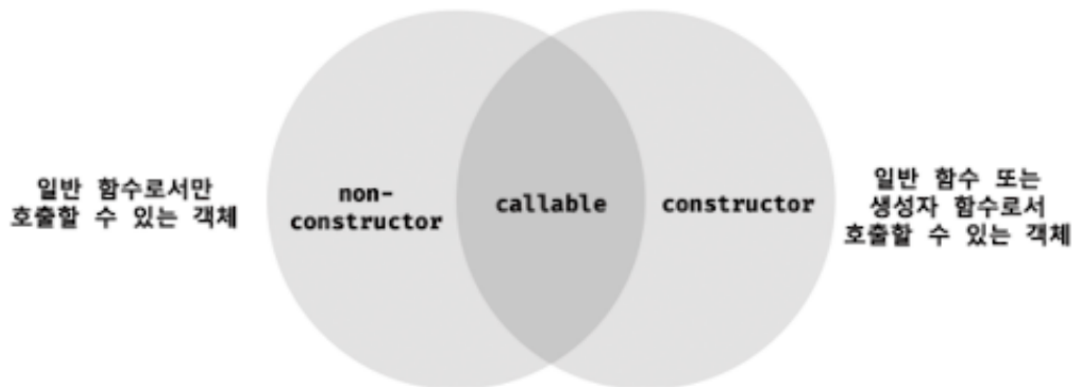
console.log(customer00);
// {name: '홍길동', ages: 30, fnCust00: f, fnCust01: f}
console.log(customer01);
```

```
// cust {name: '김길고', ages: 50, fnCust00: f, fnCust01: f}

console.log(customer00.fnCust00());
// fnCust00 :: 홍길동님 나이는 30 입니다
console.log(customer01.fnCust01());
// fnCust01 :: 김길고님 나이는 50 입니다
```

2-3-1. new로 생성하는 것과 아닌것의 차이

모든 객체는 Callable 이지만 모든 함수 객체가 constructor인 것은 아니다.



constructor와 non-constructor로 구분한다.

- constructor: 함수 선언문, 함수 표현식, 클래스(클래스도 함수다)
- non-constructor: 메서드(ES6 메서드 축약 표현), 화살표 함수

일반 함수로 취급 되므로 this.xxx는 모두 호이스팅이 되고 전역화 됩니다.

```
function cust(name, ages) {
  this.name = name;
  this.ages = ages;
  this.fnCust_00 = () => {
    return `fnCust_00 :: ${this.name}님 나이는 ${this.ages} 입니다`;
  };
  this.fnCust_01 = function() {
    return `fnCust_00 :: ${this.name}님 나이는 ${this.ages} 입니다`;
  };

  return "고객";
}
```

// 일반 함수로 취급 되므로 this.xxx는 모두 호이스팅이 되고 전역화 됩니다.

```
const fnCust001 = cust('홍길동', 20);
console.log(fnCust001); // 고객
console.log(name); // 홍길동
console.log(ages); // 20
console.log(fnCust_00()); // fnCust_00 :: 홍길동님 나이는 20 입니다
console.log(fnCust_01()); // fnCust_00 :: 홍길동님 나이는 20 입니다
```

```
const fnCust002 = cust('김길고', 40);
console.log(fnCust002);    // 고객
console.log(name);        // 김길고
console.log(ages);        // 40
console.log(fnCust_00()); // fnCust_00 :: 김길고님 나이는 40 입니다
console.log(fnCust_01()); // fnCust_00 :: 김길고님 나이는 40 입니다

console.log(fnCust001 === fnCust002) // true
```

2-4. 클래스 생성

지바스크립트 클래스는 class 표현식 and class 선언 두 가지 방법을 제공하며 constructor에 매개변수를 담아 매개변수들을 통해 프로퍼티와 프로퍼티 메소드를 만듭니다.

함수 선언과 클래스 선언의 중요한 차이점은 함수의 경우 정의하기 하기 전에 호출할 수 있지만, 클래스는 반드시 정의한 뒤에 사용할 수 있다는 점입니다

2-4-1. 클래스 선언

```
class 클래스명 {
  constructor(매개변수1, 매개변수2, ...){
    this.프로퍼티1 = 매개변수 값;
    this.프로퍼티2 = 매개변수 값;
    ...
  }
  메소드명(매개변수1, 매개변수2, ...){
    수행할 문장
    ...
  }
  ...
}
```

2-4-2. 클래스 표현식

```
const 클래스명 = class {
  constructor(매개변수1, 매개변수2, ...){
    this.프로퍼티1 = 매개변수 값;
    this.프로퍼티2 = 매개변수 값;
    ...
  }
  메소드명(매개변수1, 매개변수2, ...){
    수행할 문장
    ...
  }
  ...
}
```

2-4-3. 예제

```

let Customer = class {
  //constructor로 생성자
  constructor(name, age) {
    //this.은 프로퍼티의 부모를 말한다.
    this.name = name;
    this.age = age;

    // 익명함수로 생성할 수도 있고
    this.fnHelloMsg = function(){
      console.log(this.name + "님 나이는" + this.age );
    }
  }

  //기명함수로 생성할 수도있다.
  fnByeBye(){
    console.log(this.name + "님 안녕 ~~~~!" );
  }
}

// 객체 생성
const cust = new Customer("홍길동", 30);
console.log(cust);

/**
 * Customer { name: '홍길동',
 *           age: 30,
 *           fnHelloMsg: f }
 * [[Prototype]]: Object
 * constructor: class Customer
 * fnByeBye: f fnByeBye()
 * [[Prototype]]: Object
 *   constructor: f
 *   ....
 */

cust.fnHelloMsg(); // 홍길동님 나이는30
cust.fnByeBye();  // 홍길동님 안녕 ~~~~!

const cust01 = new Customer("김길동", 40);
cust01.fnHelloMsg(); // 김길동님 나이는40

console.log(cust === cust01); // false
console.log(cust === cust);   // true

```

3. 불변 객체

전달 받은 객체가 변경이 되어도 원본 객체는 변하지 말아야 할 때 불변 객체를 만듭니다.

3-1. Shallow Copy

```

var obj = {
  a: 1,
  b: '문자'
}

var fChgObj = function(obj, b) {
  var obj2 = obj;
  obj2.b = b;
  return obj2
}

var obj3 = fChgObj(obj, '숫자');
console.log(obj === obj3)    // true

obj3.a = 2;
console.log(obj) // {a: 2, b: '숫자'}
console.log(obj3) //{a: 2, b: '숫자'}

```

- 12 라인 : fChgObj 함수로 obj3 객체를 생성 합니다.
- 7라인 : obj를 obj2에 shadow copy
- 13 ~ 17 라인 : obj 와 obj3 는 동일 객체이므로 obj3에서 값이 변경 되면 obj 도 같이 변경됩니다.

3-2. Deep Copy

3-2-1. 새로운 객체를 만들어서 반환 해라

```
{% code lineNumbers="true" %}
```

```

var obj = {
  a: 1,
  b: '문자'
}

var fChgObj = function(obj, b) {
  // 새로운 객체로 반환
  return {
    a: obj.a,
    b: b
  }
}

var obj3 = fChgObj(obj, '숫자');

console.log(obj === obj3); // false
console.log(obj) // {a: 1, b: '문자'}
console.log(obj3) // {a: 1, b: '숫자'}

```

```
{% endcode %}
```



```
{% code lineNumbers="true" %}
```

```
const sourceObj = {
  a: 1,
  b: '홍길동'
}

// 깊은 복사(deep copy)
const fnCopy = function(pSourceObj) {
  // 새로운 객체로 반환
  const result = {};
  for( let prop in pSourceObj) {
    result[prop] = sourceObj[prop];
  }
  return result;
}

const targetObj = fnCopy(sourceObj);
console.log(sourceObj === targetObj); // false

console.log(sourceObj); // {a: 1, b: '홍길동'}
console.log(targetObj); // {a: 1, b: '홍길동'}

targetObj.b = '김길동';
targetObj.a = '2';

// sourceObj 의 값은 변경이 되지 않는다.
console.log(sourceObj); // {a: 1, b: '홍길동'}
console.log(targetObj); // {a: '2', b: '김길동'}
```

```
{% endcode %}
```

3-2-2. 내부 객체는 Shallow Copy 이다.

```
{% code lineNumbers="true" %}
```

```
var obj = {
  a: 1,
  b: { a:1, b: '문자' }
}

var shallowCopy = function(target) {
  let copy = {};
  for( var prop in target) {
    // 객체는 주소 참조 발생
    copy[prop] = target[prop];
  }
  return copy;
}
```

```

var dobj = shallowCopy(obj);
dobj.a = 2;
obj.b.a = 2;
dobj.b.b = '숫자';

console.log(obj);          // { a: 1, b: { a: 2, b: '숫자' } }
console.log(dobj);         // { a: 2, b: { a: 2, b: '숫자' } }
console.log(obj.a === dobj.a ); // false
console.log(obj.b === dobj.b ); // true
console.log(obj.b.a === dobj.b.a ); // true
console.log(obj.b.b === dobj.b.b ); // true

```

```
{% endcode %}
```

- 9 라인 : private type은 주소 분리가 되지만 객체는 주소 복사가 발생 합니다.
- 15 ~ 17 라인 : 값 변경
- 22 라인 : a 속성은 틀린 값
- 23 라인 : 객체 b 는 동일 주소로 값이 같이 변경되게 됩니다. (obj.b, dobj.b)

객체는 객체를 생성 해서 넘기기 위해 재귀 호출을 하여 생성해야 합니다.

```
{% code lineNumbers="true" %}
```

```

var obj = {
  a: 1,
  b: { a:1, b: '문자' }
}

var deepCopy = function (target) {
  let copy = {};
  // object 이면 객체를 생성 해서 넘기기 위해 재귀 호출을 하여 생성해야 합니다.
  if ( typeof target === 'object' && target !== null ) {
    for (let prop in target) {
      copy[prop] = deepCopy(target[prop]);
    }
  } else {
    copy = target;
  }

  return copy;
}

var dobj = deepCopy(obj);
dobj.a = 2;
obj.b.a = 2;
dobj.b.b = '숫자';

console.log(obj);          // { a: 1, b: { a: 2, b: '문자' } }
console.log(dobj);         // { a: 2, b: { a: 1, b: '숫자' } }
console.log(obj.a === dobj.a ); // false

```

```
console.log(obj.b.a === dobj.b.a ); // false  
console.log(obj.b.b === dobj.b.b ); // false
```

{% endcode %}